

Twitter For Jokes – dokumentace

Obsah

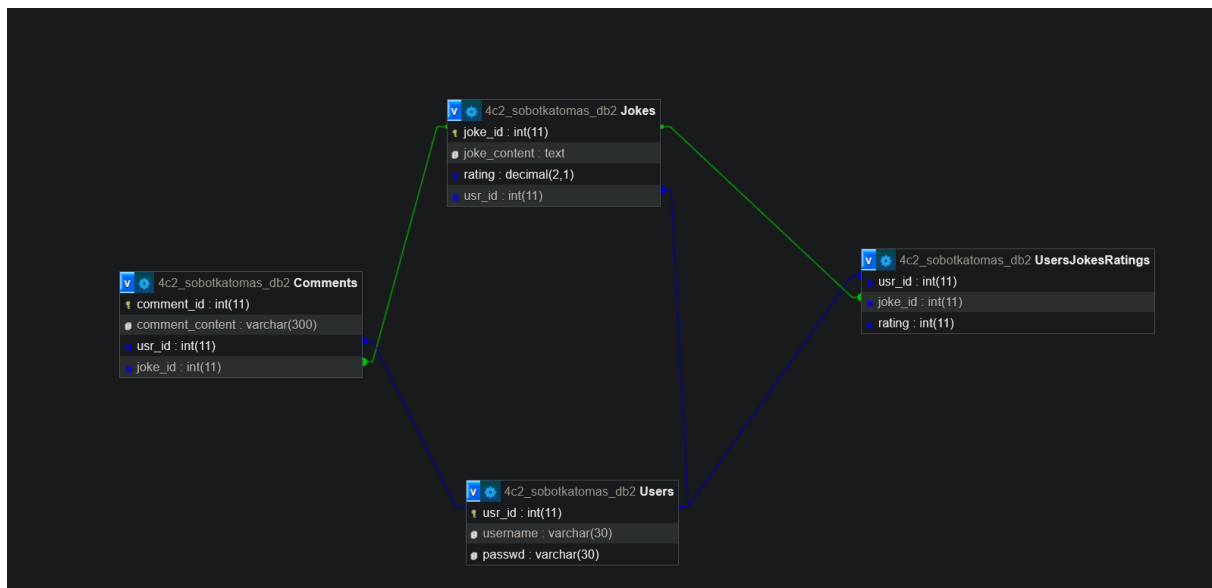
1. Backend	1
1.1. Databáze.....	1
1.2. ASP.NET Core API	3
1. Entity	3
2. Napojení na databázi.....	4
3. DTO třídy	5
4. Controllery	6
5. Další části backendu	8
2. Frontend	9
2.1. Struktura	9
2.2. Stránky.....	10
2.3. Komponenty.....	11
Efekty	12
Komponenta AuthContext	13

1. Backend

Backendová část se skládá ze dvou částí, které řeší datovou logiku v rámci aplikace a celé architektury. Tyto dvě části se skládají z databáze a ASP.NET Core API.

1.1. Databáze

Databáze se skládá ze tří entit – User (v API jako Usr), Joke, Comment – z nichž tabulka pro entitu User je nadřazená.



Snímek 1

Jakožto databázový server slouží naše školní rozhraní phpMyAdmin za použití vestavěného jazyka pro tvorbu databází MySQL, především kvůli jeho dostupnosti i ve školním prostředí. Jisté nedokonalosti a úpravy tabulky (jako mazání některých částí databáze, mazání zastaralých záznamů, integritní omezení) museli být implementovány ručně a byli následně upravováni i za běhu vývoje aplikace.

V průběhu vývoje vznikla samostatná tabulka UsersJokesRatings pro splnění požadavků na systém hodnocení vtipů. Tato tabulka tvoří vztah M:N mezi tabulkami Users a Jokes a dovoluje pro každého uživatele mít jedinečný rating na každém vtipu. Průměrné hodnocení pro každý vtip už v databázi implementováno není, ale je součástí webového API.

1.2. ASP.NET Core API

Backend poskytuje HTTP API a posílá JSON soubory, které následně využívá frontend pro zobrazení dat uživateli. V projektu jsou zahrnuty 4 hlavní části kódu:

- Entity
- Napojení na databázi
- DTO třídy
- Controllery

1. Entity

Entity v API odpovídají přesně počtu tabulek, které jsou v databázi, tedy i tabulka `UsersJokesRatings` figuruje jako samostatná entita. Pro každou entitní třídu bylo použito náležitého namapování na sloupce v tabulkách, tak aby data byla konzistentní a funkční.

Např. entita `Usrs`:

```

using MySqlX.Crud;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace TwitterForJokes.Entities
{
    [Table("Users")]
    Počet odkazů: 14
    public class User
    {
        [Key]
        [Column("usr_id")]
        Počet odkazů: 7
        public int UserId { get; set; }
        [Column("username")]
        Počet odkazů: 10
        public string UserName { get; set; }
        [Column("passwd")]
        Počet odkazů: 3
        public string Password { get; set; }

        Počet odkazů: 3
        public ICollection<Joke> Jokes { get; set; } = new List<Joke>();

        Počet odkazů: 1
        public User() { }

        Počet odkazů: 0
        public User(int userId, string userName, string password)
        {
            UserId = userId;
            UserName = userName;
            Password = password;
        }
    }
}

```

Snímek 2

V závislosti na snímku 2 je třeba poznamenat, že konstruktory nejsou v entitních třídách totožné a snímek je v tomto ohledu nepřikladný. Konstruktory jsou vytvořené každý jinak a každý zvlášť podle individuálních nutností.

2. Napojení na databázi

Webové API se připojuje k databázovému serveru skrz třídu `AppDbContext` ve které jsou všechny `DbSets` pro každou tabulku v databázi. Tato třída dědí z třídy `DbContext`.

```

using Microsoft.EntityFrameworkCore;
using TwitterForJokes.Entities;

namespace TwitterForJokes.Context
{
    Počet odkazů: 13
    public class AppDbContext : DbContext
    {
        Počet odkazů: 10
        public DbSet<User> Users { get; set; }
        Počet odkazů: 9
        public DbSet<Joke> Jokes { get; set; }
        Počet odkazů: 6
        public DbSet<Comment> Comments { get; set; }
        Počet odkazů: 5
        public DbSet<UsersJokesRating> UsersJokesRatings { get; set; }

        Počet odkazů: 0
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
    }
}

```

Snímek 3

Z důvodu bezpečnosti projektu a sdílení verzování práce na veřejný GitHub repozitář jsme byli donuceni připojovat se do databáze pomocí tajných klíčů uživatelů, aby connection string, který je podmínkou spojení mezi oběma konci, nebyl dostupný veřejně. Tyto klíče jsou uloženy lokálně mimo repozitář.

Aby tento způsob mohl fungovat, program musí tuto informaci dostat a zpracovat ve třídě program.cs:

```
var connString = builder.Configuration.GetConnectionString("uplne_nejvic_tajny_spojovaci_klic");

builder.Services.AddCors(options =>
{
    options.AddPolicy("allow-all", policy =>
        policy.WithOrigins("http://localhost:5173")
            .AllowAnyMethod()
            .AllowAnyHeader()
            .AllowCredentials()
    );
});

builder.Services.AddDbContext<AppDbContext>(options => options.UseMySQL(connString));
```

Snímek 4

3. DTO třídy

U velké většiny akcí v controllerech se práce s daty neodehrává přímo na entitních třídách, ale na DTO (data transfer object) třídách. DTO třídy osekávají celé entity a není tak potřeba vždy v POST, GET nebo PUT akcích v JSON souboru posílat záznamy jako celou entitu. Tento přístup je mnohdy výhodný a někdy i neojediněle nutný.

Typicky jde například o Id (jedinečný identifikátor) instance/záznamu, kdy nechceme, aby s ním uživatel interagoval, jako můžeme vidět ve třídách CreateUserDto nebo CreateJokeDto. Může se ale také jednat jen o zjednodušený dataset, který chceme na frontend posílat a nemusí jít přímo o interní logiku interakce prostředí s uživatelem. Většina akcí v controllerech je řízena přesně pomocí těchto tříd.

Typické DTO v API:

```

namespace FilterProbes.DTOEntities
{
    /// <summary>
    /// DTO entities for creation restrictions during HttpPost API calls
    /// To transfer just part of the entity from UI (user interface)
    /// </summary>
    Počet odkazů: 1
    public class CreateJokeDto
    {
        /// <summary>
        /// TEMPORARY attribute
        /// Is going to be deleted and unavailable to create and transfer from user interface in future
        /// </summary>
        //public int UsrcId { get; set; }
        Počet odkazů: 1
        public string JokeContent { get; set; }
        Počet odkazů: 1
        public decimal Rating { get; set; }
    }
}

```

Snímek 5

4. Controllery

Pro každou entitu, byl vytvořen jeden vlastní controller. V programu se nachází ještě controller pro autentikaci a práci nad přihlašovacími daty – `AuthenticationController`.

V každém controlleru jsou implementovány akce pro jednotlivé činnosti programu, které „obslouží“ uživatele, stejně tak ale mohou být přínosné pro vývojáře (např. kontrola správného zasílání dat).

Pro každý controller je ještě zaveden databázový kontext, pro komunikaci s databází s inicializací v konstruktoru typu `private readonly`.

Příklad controlleru v API:

```

namespace TwitterForJokes.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    Počet odkazů: 1
    public class JokesController : Controller
    {
        private readonly AppDbContext _context;

        Počet odkazů: 0
        public JokesController(AppDbContext context)
        {
            _context = context;
        }

        [HttpGet]
        Počet odkazů: 0
        public async Task<ActionResult<List<Joke>>> GetAllJokes()
        {
            var allJokes = await _context.Jokes.Select(j => new GetJokeDto
            {
                JokeId = j.JokeId,
                JokeContent = j.JokeContent,
                Rating = j.Rating.ToString(new CultureInfo("cs-CZ")),
                AuthorName = j.User!.UserName
            })
            .ToListAsync();

            return Ok(allJokes);
        }
    }
}

```

Snímek 6

Na každém konstruktoru jsou endpointy, které definují cestu pro frontend a vytažení konkrétních dat v tomto formátu:

```

[ApiController]
[Route("api/[controller]")]

```

Snímek 7

Uvnitř akcí jsou zabudovány validační podmínky, které kontrolují a vypisují chybové hlášky, pokud nastane problém při jejich spouštění. Tyto podmínky většinou nemají vliv na vývoj frontendu, nýbrž slouží pro testování funkčnosti backendu ve výchozím klientovi Swagger. Jsou vytvořeny tedy především pro pohodlnější vývoj backendové části architektury.

```

[HttpPut("{id}/rating")]
Počet odkazů: 0
public async Task<ActionResult> EditJokeRating(int id, EditJokeRatingDto dto)
{
    var joke = await _context.Jokes.FindAsync(id);

    if (joke == null)
    {
        return NotFound("This joke does not exist bro.");
    }

    if (dto.Rating < 0 || dto.Rating > 10)
    {
        return BadRequest("Rating must be in required format.");
    }

    joke.Rating = dto.Rating;

    await _context.SaveChangesAsync();
    return Ok(joke);
}

```

Snímek 8

5. Další části backendu

Webové API obsahuje více částí. Prominentně se bavíme o hlavní třídě `program.cs` a implementaci JWT tokenů pro přihlašování.

Soubor `program.cs` dále obsahuje:

- konfigurace JWT autentizačního middlewaru – `AddAuthentication`, `AddJwtBearer`
- `HttpOnly` cookies
- aktivace CORS – `AddCors`

V našem API se nachází třída `TokenProvider`, která obsahuje metodu, jež jednoznačně definuje JWT token a jeho claims a vytvoří ho. Token je podepsán pomocí algoritmu `HmacSha256`.

`TokenProvider`:

```
Počet odkazů: 3
public sealed class TokenProvider(IConfiguration configuration)
{
    Počet odkazů: 1
    public string Create(Usr user)
    {
        string secretKey = configuration["Jwt:Secret"]!;
        var securityKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(secretKey));

        var credentials = new SigningCredentials(securityKey, SecurityAlgorithms.HmacSha256);

        var tokenDescriptor = new SecurityTokenDescriptor
        {
            Subject = new ClaimsIdentity([
                new Claim(JwtClaimNames.Sub, user.UserId.ToString()),
                new Claim(JwtClaimNames.UniqueName, user.UserName.ToString())
            ]),

            Expires = DateTime.UtcNow.AddMinutes(configuration.GetValue<int>("Jwt:MinuteExpiration")),
            SigningCredentials = credentials,
            Issuer = configuration["Jwt:Issuer"],
            Audience = configuration["Jwt:Audience"]
        };

        var handler = new JwtTokenHandler();
        string token = handler.CreateToken(tokenDescriptor);

        return token;
    }
}
```

Snímek 9

2. Frontend

Frontend pro projekt Twitter For Jokes byl vyvíjen za pomoci knihovny React a programovacího jazyka TypeScript. Tento produkt byl vybrán z důvodu naplnění všech potřeb pro plnohodnotnou webovou aplikaci.

2.1. Struktura

Struktura frontendové části celé aplikace se dělí na tři základní složky:

- Assets – obrázky, gify
- Modules – jednotlivé komponenty a části kódu pro zobrazení
- Pages – jednotlivé stránky (kromě stránky Jokes)

Mimo všechny tyto složky ještě působí soubor `App.tsx`, `Main.tsx` a `Main.css`, které zaopatřují vlastnosti platné pro celou aplikaci. V rámci souboru `App.tsx` je zabudováno mapování cest jednotlivých stránek (URL adresy) – Routes. Je tak globálně umožněno „cestovat“ mezi stránkami, které v aplikaci jsou.

Pro element odkazu (Link) poté stačí v konkrétním místě v kódu jen napsat správný redirect.

```
function App() :Element { Show usages  & gest2356 +3
  return (
    <>
      <AuthProvider>
        <Toaster position="top-left" reverseOrder={false} />

        <NavBar />
        <div id="snowfall-effect">
          <Snowfall style={{position: "fixed",
            width: "100vw",
            height: "100vh",
            zIndex: 1,
            pointerEvents: "none",}} color="white" snowflakeCount={100} />
        </div>

        <Routes>
          <Route path="/" element={<Landing />} />
          <Route path="/profile" element={<Profile />}></Route>
          <Route path="/feed" element={<Feed />} />
          <Route path="/creators" element={<Creators />} />
          <Route path="/playground" element={<PlayGround />} />

          <Route path="/login" element={<Login />} />
          <Route path="/register" element={<Register />} />

          <Route path="/jokes/:id" element={<JokeDetailPage />} />

        </Routes>
      </>
    </>
  )
}
```

Snímek 10

Pro stylování je u každého souboru (tam kde to má smysl) vytvořený css soubor, kde jsou zpracovány adresované styly pouze pro specifický obsah (komponenta, stránka), pojmenovaný totožně jako soubor tsx.

2.2. Stránky

Stránky neobsahují logiku práce s daty (kromě stránky [JokeDetailPage.tsx](#)), které dostáváme od backendu – fetch dotazy a vrácení html obsahu je předmětem kódu v komponentách. Na stránkách se pouze vloží prvky komponent do prázdné jednoduché funkce. Jejich logika je ale rozpracována jinde. Takové paradigma se nám zdálo intuitivnější a přehlednější s co nejmenším rizikem nepochopení jednotlivých úkonů částí kódu.

Příklad tsx souboru jako stránka:

```
import ...

function Login() : Element { Show usages ⓘ TrueTexar +1

    return (
        <>
        <LoginBubble></LoginBubble>
        </>
    )
}

export default Login; Show usages ⓘ TrueTexar
```

Snímek 11

Příkladně podle snímku 11, je zde jen funkce, která pouze vrátí jinde složitěji naprogramovanou komponentu.

2.3. Komponenty

Komponenty uchovávají komunikaci s daty a backendem, se kterými chceme operovat a zobrazují jen určitou část kódu. V těchto souborech se také mohou vyskytovat další komponenty, které typicky jsou potřebné pro místní kód, respektive komponenta potřebuje další komponentu jen v této komponentě ať už z jakéhokoliv důvodu.

Tuto vlastnost splňuje například komponenta `CommentBubble`:

Komponenta v komponentě `CommentBubble` `ConfirmModal`:

```
const ConfirmModal : ({ message, onConfirm, onCancel }: ConfirmModalProps) : Element => {  
  return (  
    <>  
      <div className="modal-overlay">  
        <div className="modal-box">  
          <p>{message}</p>  
  
          <div className="modal-buttons">  
            <button id="first-button" onClick={onCancel}>Cancel</button>  
            <button id="second-button" onClick={onConfirm}>Delete</button>  
          </div>  
        </div>  
      </div>  
    </>  
  )  
}
```

Snímek 12

Return v komponentě `CommentBubble` obsahující také `ConfirmModal`:

```
return (  
  <>  
    {isConfirmOpen &&  
      <ConfirmModal  
        message={"Do you really want to delete this comment?"}  
        onConfirm={confirmDelete}  
        onCancel={() :void => setIsConfirmOpen( value: false)}  
      />  
    }  
  </div>  
  <h1 id="comment-header">All comments:</h1>  
  {data.map((comment: Comment) :Element => (  
    <div className="bubble-div" key={comment.commentId}>  
      <div>  
        <img src={usr_icon} alt="" />  
        <p>{comment.authorName}</p>  
      </div>  
      <CommentText text={comment.commentContent}></CommentText>  
      { comment.authorName === user.user?.userName || user.user?.userName === 'AdminAdmin'?  
        <button onClick={() :void => {  
          setIsConfirmOpen( value: true);  
          setSelectedCommentId(comment.commentId);  
        }}>Delete comment</button>  
        : null  
      }  
    </div>  
  ) )  
  </div>  
)  
)
```

Snímek 13

Fetch dotazy probíhají vždy v těle funkce. V případě, že nastane chyba na straně serveru a přenos dat skutečně nefunguje správně je uživatel na stránce informován pomocí vyskakovacího okna (toast).

Například komponenta `JokeBubble` při snaze smazat vtip:

```
try {
  const res :Response = await fetch( input: `http://localhost:65451/api/Jokes/${id}`, {
    method: 'DELETE',
  });

  if (res.ok) {
    setMessage( value: "Joke was deleted successfully.");
    refreshJokes();
    toast.success( message: "Your joke was successfully deleted!");
  } else {
    const text :string = await res.text();
    setError(text || "Unable to delete the joke.");
    toast.error( message: "there was en error while deleting the joke.");
  }
} catch {
  setError( value: "Network error occurred while deleting.");
}
```

Snímek 14

Vyskakovací okna jsou v aplikaci jako rychlý, dynamický a efektivní ukazatel informací o akcích, které vykonává. Proto jsou zde vytvořené. Tyto vyskakovací okna však zatím nejsou implementovány úplně všude a někde prozatím mohou chybět.

Vyskakovací okna jsou také předavatelem informací po úspěšném provedení akce.

Efekty

Pro doplnění aplikace obsahuje vizuální efekty, což jsou dodatečně stáhnuté balíčky do knihovny. V naší aplikaci jsou dva typy efektů – globální, lokální.

Globální efekty platí pro celou webovou aplikaci. Lokální platí jen pro určitou část webu a spustí se jen v speciálním případě.

Například firework effect v komponentě `LoginBubble` je příklad lokálního efektu:

```
const {reward} = useReward(logInButton as React.RefObject<HTMLInputElement>, {
  animationType: 'fireworks',{
    particleCount: 90,
    spread: 9,
    colors: ['#ff0000', '#00ff00', '#0000ff'],
  });
```

Snímek 15

Komponenta AuthContext

Tato komponenta je implementována pro globální správu autentizace uživatele pomocí React Context. Slouží k tomu, aby celá aplikace věděla, zda-li je uživatel přihlášený a kdo to je. Díky AuthProvideru mají komponenty (a celá knihovna) přístup k přihlášenému uživateli, odhlašování a kontrole přihlášení.

Data která se vrátí jsou definovaná na backendu, proto interface uživatele v této komponentě uchovává právě tyto data:

```
interface User { Show usages ⓘ TrueT
  isAuthenticated: boolean;
  userId: number;
  userName: string;
}
```

Snímek 16

Vlastnost isAuthenticated je zejména důležitá. Podle ní je uchováváno v zobrazování html obsahu mnoho podmínek.

Např [CommentPostBubble](#):

```
if (user.user?.isAuthenticated === true) {
  return (
    <>
      <div className="whole-comment-post">
        <h1>Comment this joke!</h1>
        <form className="comment-post-form" onSubmit={postComment}>
          <p>Your comment:</p>
          <textarea className="post-comment-input" onChange={(e: ChangeEvent<HTMLTextAreaElement>)} :void => setCommentContent(e.target.value)} value={commentContent} />
          <button type="submit">Post comment</button>
        </form>
      </div>
      {loading && <Loading />}
    </>
  )
}
else {
  return (
    <>
      <div className="whole-comment-post">
        <p id="nothing-to-comment-message">To comment this joke you need to log in.</p>
      </div>
    </>
  )
}
```

Snímek 17

Na snímku 17, pokud je uživatel přihlášen, má přístup ke značně jiným funkcím a zobrazení, než uživatel nepřihlášený.

T.B. Sobotka

V. Javůrek

